

FE-APP-00 - Frontend Architecture and Shared UX Foundation

Version: v1.0

Status: Draft for review

Depends on:

- DD_CROSS-00-System_Traceability_and_Implementation_Map-v1.0
- DD_API-00-API_Catalog_and_Gateway_Standards-v1.0
- DD_QA-01-End_to_End_Test_Strategy-v1.0
- FOUNDATION_AUTH
- FOUNDATION_FILE_STORAGE

Applies to:

- FE-APP-01 BSS Backoffice Web App
- FE-APP-02 Outdoor Sales / Franchise App
- FE-APP-03 Contractor Technical Mobile App
- FE-APP-04 Customer Self-Care Frontoffice App
- FE-APP-05 Reporting Portal

1. Purpose

This DD defines the shared frontend foundation for SOPHIX V3.

It explains how all frontend applications should be built so that they are consistent, secure, operator-flexible, and practical to maintain. It does not design every screen. Each application-specific DD will define screens and flows for its own users.

SOPHIX is deployed in Kenya first, but the frontend must support future deployment into other regions with minor configuration changes. The same codebase must support different operators, countries, languages, currencies, logos, color themes, feature availability, and local compliance text.

Every frontend app DD must also satisfy DD_CROSS-00-System_Traceability_and_Implementation_Map-v1.0: each feature/screen must identify the owning module API, supporting read APIs, role source, events/polling behavior, and E2E test scenario.

2. Frontend Application Set

We will document six frontend DDs:

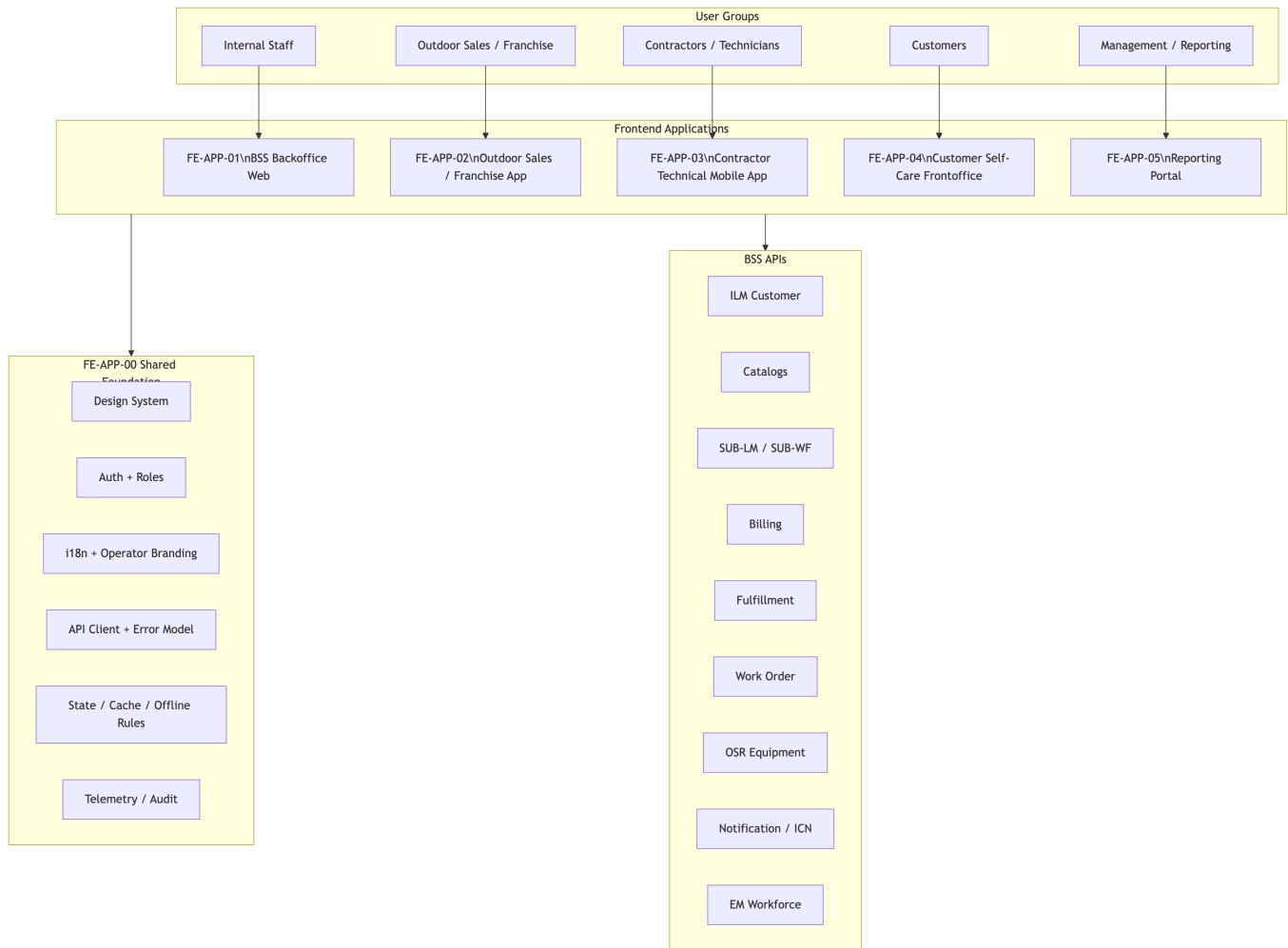
DD	Application	Users	Deployment note
FE-APP-00	Frontend Architecture and Shared UX Foundation	Frontend/backend devs, UX, QA, architects	Shared rules, not a standalone user app.
FE-APP-01	BSS Backoffice Web App	Internal operations, customer care, billing, fulfillment, dispatch, catalog/admin teams	One internal web app with role-based modules.
FE-APP-02	Outdoor Sales / Franchise App	Door-to-door agents, franchise sales teams, sales supervisors	Mobile-first PWA/app using SALES-01 for leads/territory/attribution and FUL-02 for submitted orders.
FE-APP-03	Contractor Technical Mobile App	Contractors, field technicians, team leads	Mobile-first app, offline job execution and evidence capture.
FE-APP-04	Customer Self-Care Frontoffice App	End customers	Customer portal/app for account, billing, support, limited self-service.
FE-APP-05	Reporting Portal	Management, finance, operations, supervisors	Uses REP-01 dashboard/export APIs; can start inside Backoffice shell if budget requires.

Implementation readiness rule: do not start a frontend screen until its owning backend module API is identified in the app DD and in DD_CROSS-00. The API must follow DD_API-00; important screens must map to an E2E scenario from DD_QA-01 or

add one.

Illustration:

Figure 1 - Frontend application landscape



3. Design Principles

Principle	Meaning
One codebase, many operators	Avoid country forks. Operator differences must be configuration, translations, theme tokens, and feature flags.
Module ownership remains backend-owned	UI screens call owning module APIs. Frontend does not create hidden ownership rules.
Role-driven navigation	Users see only actions allowed by their roles and operator configuration. Backend still enforces authorization.
Explicit process state	Long-running operations show current state, next expected action, owner, timers, and failure reason.
Mobile workflows survive weak networks	Sales and contractor apps need drafts, offline queues, retry, and conflict handling.
Operational density for backoffice	Backoffice is a work tool. Use compact, scannable screens rather than marketing-style layouts.
Customer clarity for frontoffice	Customer-facing screens use simpler language and hide internal workflow complexity.
Accessibility by default	Support keyboard navigation, readable contrast, labels, focus states, and responsive layouts.

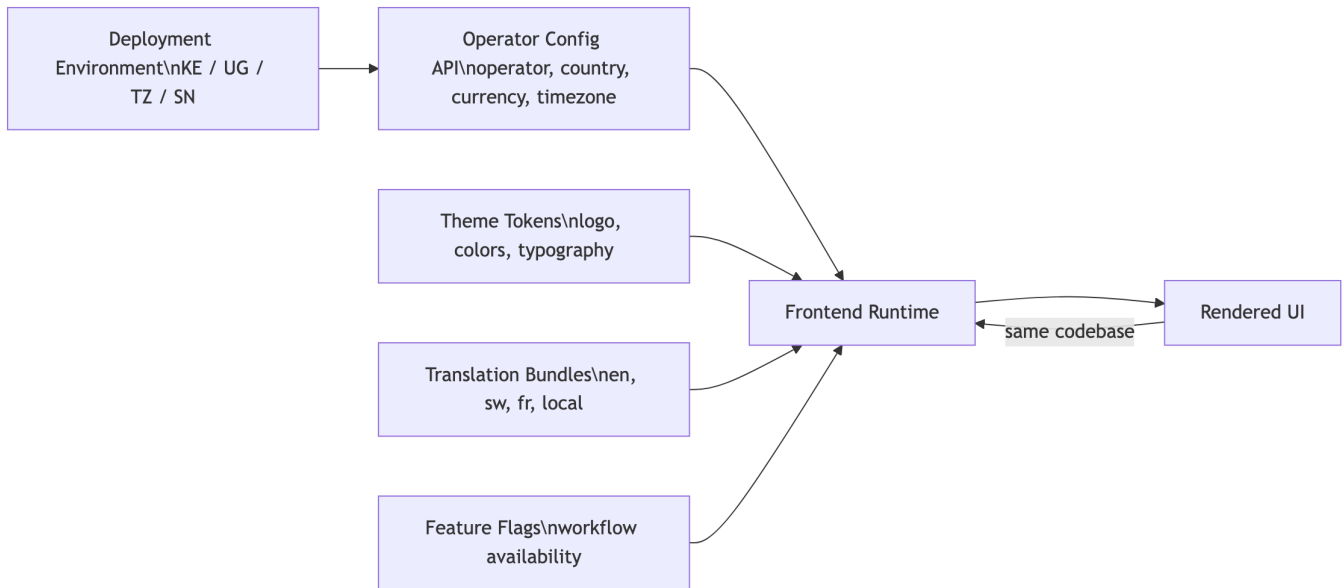
4. Regional And Operator Flexibility

Kenya is the first deployment. The frontend must also support future regions such as Uganda, Tanzania, Senegal, and other operator brands without a code fork.

4.1 Runtime Configuration Model

Illustration:

Figure 2 - Runtime configuration for regional flexibility



Every frontend app loads runtime configuration before rendering protected business screens.

Minimum runtime config:

Config area	Example values	Used for
Operator	WIK, WUG, WTZ, YASSN	API scoping, labels, feature flags, brand.
Country	KE, UG, TZ, SN	Address format, tax labels, language defaults, currency.
Locale	en-KE, sw-KE, fr-SN	Dates, numbers, text translations.
Currency	KES, UGX, TZS, XOF	Money display and form validation.
Timezone	Africa/Nairobi, Africa/Kampala, Africa/Dar_es_Salaam, Africa/Dakar	Date/time display and scheduling.
Brand	Logo URL, primary color, app name	Header, login page, icon, PDF/report branding.
Feature flags	walletEnabled, taxInvoiceEnabled, selfPauseEnabled	Hide/show flows without code changes.
Legal text	T&C version, privacy links, consent text	Sales, self-care, KYC, payment.

4.2 Theme Rules

Use design tokens, not hardcoded colors.

Required theme token groups:

Token group	Examples
Brand	brand.primary, brand.secondary, brand.logoUrl, brand.appName
Status	status.success, status.warning, status.danger, status.info
Layout	spacing scale, border radius, density
Typography	font family, size scale, weight scale
Data visualization	chart palette with contrast-safe colors

Rule: the operator can change logo, primary colors, and app name through configuration. The application layout and component behavior must remain stable.

4.3 Language And Translation Rules

All user-visible text must come from translation keys, except dynamic values returned by APIs.

Translation key convention:

```
<app>.<domain>.<screen>.<element>
```

Examples:

```
backoffice.customer360.title  
sales.orderCapture.homepassNotEligible  
contractor.workOrder.finalizeButton  
selfcare.invoice.payNow  
reports.billing.dunningAtRisk
```

Rules:

- Do not concatenate translated strings with variables in a way that breaks grammar.
- Use ICU-style messages for pluralization and interpolation.
- Store dates, money, and numbers as raw values; format in the UI using locale/currency config.
- Backend error codes must map to localized frontend messages.
- Support left-to-right layouts initially; do not block future RTL support with hardcoded alignment assumptions.

5. Authentication And Authorization

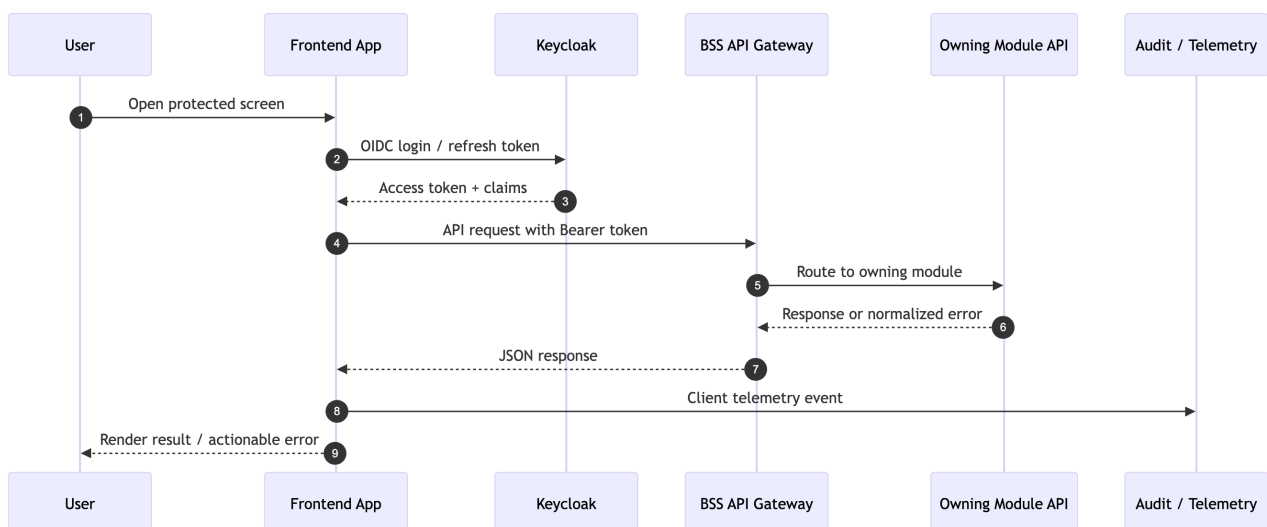
All frontend apps use Keycloak/OIDC from FOUNDATION_AUTH.

Required behavior:

1. User opens app.
2. App loads public runtime config.
3. App redirects to Keycloak if no valid session exists.
4. App receives access token and refresh token.
5. App calls APIs with `Authorization: Bearer <token>`.
6. App renders navigation/actions from token claims, role matrix, and feature flags.
7. Backend enforces authorization again on every API.

Illustration:

Figure 3 - Frontend API and auth pattern



Frontend must not trust hidden UI controls as authorization. If a user crafts a request manually, backend role guards must still reject it.

5.1 OIDC Clients

Recommended clients:

Client	App
sophix-backoffice-ui	Backoffice + CRM + Dispatch + Admin modules
sophix-sales-app	Outdoor sales/franchise
sophix-contractor-app	Contractor technical mobile
sophix-selfcare-app	Customer self-care
sophix-reporting-ui	Reporting portal, if split from backoffice

5.2 Role Handling

Frontend should use role claims for navigation and action visibility.

Examples:

Role family	Typical app access
Customer care	Customer 360, interactions, tickets, limited subscription actions.
Billing ops	Payments, invoices, adjustments, dunning, wallet.
Fulfillment ops	Orders, activation/restriction/termination status.
Dispatcher	Work order assignment, technician availability, escalation.
Field tech	Assigned work orders, installation/support actions.
Sales agent	Lead/order capture, KYC upload, payment instructions.
Catalog admin	Services, packages, HomePass, tech region, equipment catalog.
Platform admin	Users, roles, audit, system health, workflow incidents.
Reporting viewer	Dashboards, exports, read-only analytics.

6. Application Shell

Each frontend application must share a consistent shell pattern while adapting to its device and user context.

6.1 Backoffice Shell

Backoffice is a dense operational web app.

Required shell areas:

- top bar with operator switch/context if user has multi-operator access;
- left navigation grouped by module/process;
- global customer/subscription/work-order search;
- notifications/task inbox;
- profile/session menu;
- environment indicator for non-production;
- breadcrumb for deep operational screens.

6.2 Mobile Shells

Sales and contractor apps are mobile-first.

Required shell areas:

- bottom navigation for main daily actions;
- offline/connectivity indicator;
- sync status indicator;
- compact task/order/job queue;
- camera/file capture entry points where relevant;
- local draft recovery prompt.

6.3 Customer Frontoffice Shell

Customer self-care must be simpler than backoffice.

Required shell areas:

- account/subscription selector when the customer has multiple services;
- billing/payment quick actions;
- support/ticket entry point;
- notifications;
- profile/preferences.

7. API Integration Pattern

Frontend apps must call backend APIs through a shared API client layer.

Required client behavior:

Concern	Rule
Auth	Attach bearer token automatically. Refresh token before expiry.
Operator context	Send selected operator/country context where backend contract requires it.
Correlation	Send <code>X-Correlation-Id</code> for every command and important query.
Idempotency	Send <code>Idempotency-Key</code> for every create/command that may be retried.
Error mapping	Convert backend error code to localized user message and support details.
Retry	Retry safe reads with backoff. Do not blindly retry non-idempotent commands.
Timeouts	Use explicit timeouts and show actionable pending state.
Pagination	Use backend paging; do not load huge operational lists into browser memory.

7.1 Error Model

Backend errors should be displayed using a consistent frontend model.

Backend field	Frontend use
<code>errorCode</code>	Translation key and support reference.
<code>message</code>	Developer/support fallback, not always customer-safe.
<code>fieldErrors</code>	Field-level validation display.
<code>correlationId</code>	Show in details/copy button for support.
<code>retryable</code>	Controls retry button or wait message.
<code>nextAction</code>	Suggests user action where backend can provide it.

Frontend display rules:

- Use field-level errors near the field.
- Use page-level alerts for business rule rejections.
- Use modal confirmations only for irreversible or high-risk actions.
- Show correlation id in expandable details for support users.
- For customer-facing screens, avoid exposing internal module names unless needed.

8. State, Cache, And Offline Rules

Frontend local state is not a source of truth.

Allowed local storage:

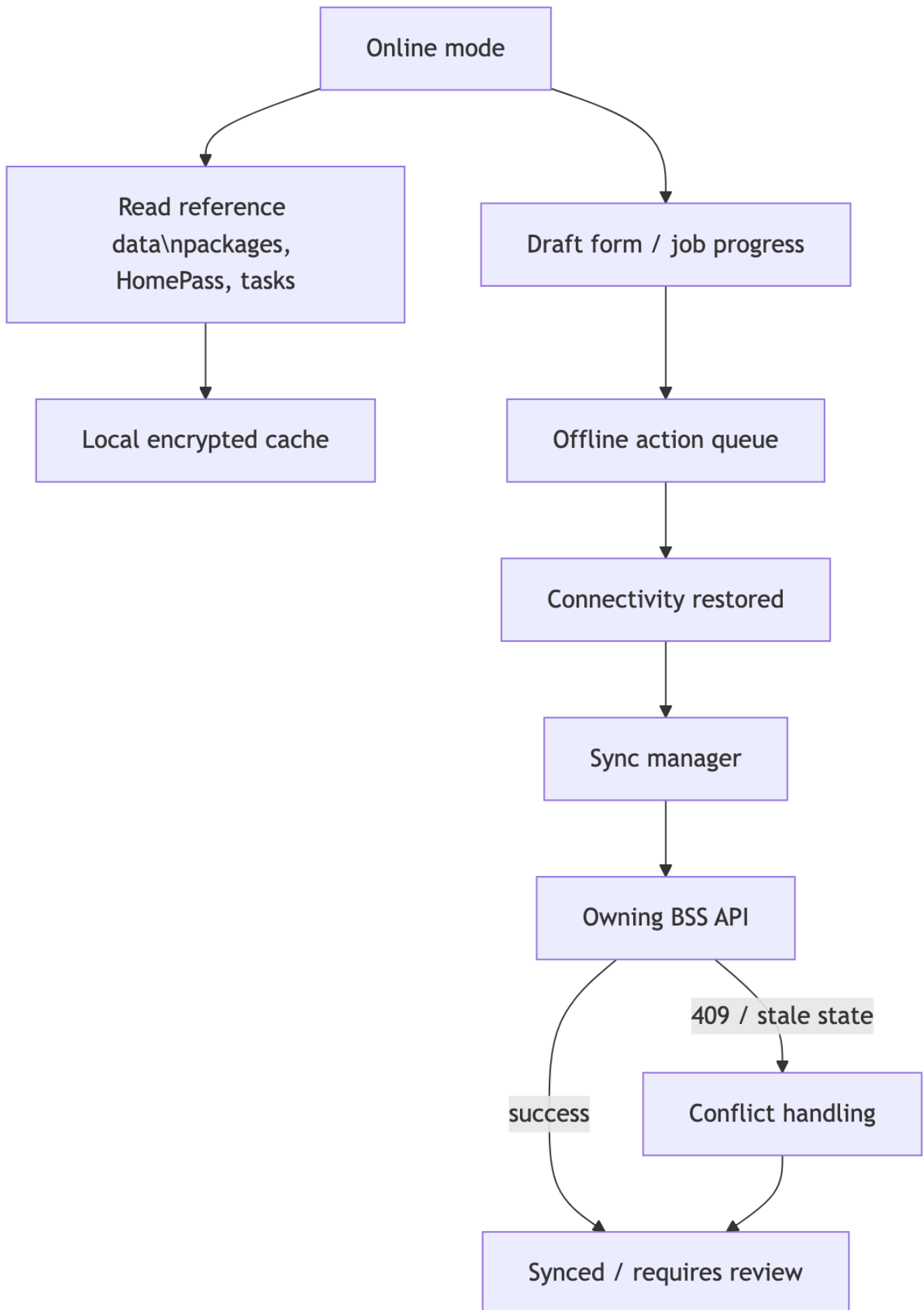
Data	Allowed?	Notes
Access token	Yes, according to selected OIDC pattern	Prefer secure storage for mobile wrappers.
Refresh token	Yes, if app security model permits	Must follow Keycloak/OIDC guidance.

Data	Allowed?	Notes
Runtime config	Yes	Refresh on app startup and periodically.
Reference data	Yes	Packages, HomePass search results, task lists; must expire.
Draft forms	Yes	Sales/order/KYC/job drafts; encrypt where possible.
Uploaded file metadata	Yes	File binary handling depends on app and device.
Authoritative financial/subscription state	No	Always refresh from owning API.

8.1 Offline Pattern

Illustration:

Figure 4 - Offline and sync pattern



Offline capability applies mainly to:

- outdoor sales/franchise app;
- contractor technical mobile app.

Offline command rules:

- Queue only commands approved by the app-specific DD.
- Every queued command must have an idempotency key.
- Show queued, syncing, synced, failed, and conflict states.
- If backend returns conflict, do not silently overwrite. Show conflict resolution flow.
- Do not allow offline financial confirmation unless the billing/payment DD explicitly supports it.

9. Forms And Validation

Frontend validation improves UX. Backend validation remains authoritative.

Required form behavior:

- validate required fields before submit;
- use backend enum/catalog APIs for select options;
- format phone numbers, IDs, dates, and money by country/operator config;
- preserve drafts for long forms;
- prevent double submit;
- attach idempotency key on create/command;
- display backend field errors without losing user input.

High-risk actions need explicit confirmation:

Action type	Confirmation requirement
Subscription termination	Required, show impact summary.
Billing adjustment/write-off/refund	Required, role-gated, show audit reason.
Work order cancellation	Required, show downstream impact.
Equipment swap/RMA finalization	Required when serial/state changes.
Customer data deletion/anonymization	Required and likely supervisor-approved.

10. File Upload And Evidence

File handling follows FOUNDATION_FILE_STORAGE.

Frontend file use cases:

App	Examples
Sales app	KYC documents, proof of address, customer photo, consent evidence.
Contractor app	Installation photos, equipment serial photos, site evidence, support visit proof.
Backoffice	Invoice PDF download, dispute attachments, admin evidence.
Customer self-care	Support attachments, possibly KYC/profile documents where enabled.

Rules:

- Do not upload directly to public object storage.
- Upload through approved module API or pre-approved private upload flow.
- Show file upload progress and retry.
- Compress photos where app-specific DD permits it.
- Preserve original metadata required for audit.

- Never put customer names, phone numbers, ID numbers, or emails in object keys.

11. Notifications And Task Inbox

Frontend notification types:

Type	Source	UI behavior
User task	Camunda/module task APIs	Show in task inbox with due date and owner.
Operational alert	Module status APIs/reporting	Show in dashboard or queue.
Customer notification	NOT	Show in customer notification history/preferences.
Staff/internal communication	ICN	Show inbox, ack/read status.
Sync status	Local app	Show queued/syncing/failed state.

Backoffice and mobile apps must distinguish between:

- notification: information for the user;
- task: something the user must act on;
- incident: failure needing operational support.

12. UX Standards

12.1 Backoffice UX

Backoffice screens should be dense, scannable, and operational.

Use:

- tables with filters, saved views, pagination;
- timeline panels for customer/subscription/work order history;
- status chips with consistent color semantics;
- split view for list + detail where workflows require fast triage;
- audit panels for high-risk records;
- role-gated action menus.

Avoid:

- marketing-style hero layouts;
- oversized cards for operational data;
- hiding important state behind decorative UI;
- actions that are visible but always fail due to permissions.

12.2 Mobile UX

Mobile apps should prioritize daily field tasks.

Use:

- clear current task/job/order state;
- large touch targets;
- camera-first upload flows;
- offline status;
- step-by-step checklists for work execution;
- background sync with visible outcome.

12.3 Customer UX

Customer self-care must simplify internal BSS terms.

Examples:

Internal term	Customer-facing label
Dunning	Payment reminder
Restriction	Limited service
Suspension non-payment	Service paused due to unpaid balance
Work order	Technician visit
Subscription operation	Request

13. Accessibility And Responsive Rules

Minimum requirements:

- WCAG AA contrast for text and interactive controls.
- Keyboard navigation for web apps.
- Visible focus states.
- Labels for all inputs.
- Error text associated with fields.
- No color-only status indicators.
- Responsive layout for desktop, tablet, and mobile where app requires it.
- Tables must have responsive alternatives for narrow screens.

14. Security Rules

Frontend must not:

- store secrets in source code;
- rely on hidden buttons for authorization;
- expose service tokens to browsers/mobile apps;
- log access tokens, refresh tokens, ID numbers, passwords, or payment secrets;
- cache sensitive documents longer than approved;
- send customer PII to third-party analytics without approval.

Frontend must:

- use HTTPS only;
- validate file type/size before upload;
- clear local state on logout;
- handle token expiry gracefully;
- display session timeout warnings where appropriate;
- include correlation id on command requests.

15. Observability

Every frontend app must emit basic telemetry.

Required events:

Event	Purpose
App loaded	Startup performance and config errors.
Login completed/failed	Auth monitoring.
API error	API reliability and frontend error mapping.
Command submitted	Correlate user action with backend operation.

Event	Purpose
Command completed/failed	Track workflow success.
Offline queue item created/synced/failed	Mobile reliability.
Upload started/completed/failed	Evidence/KYC reliability.

Telemetry must include:

- app name and version;
- operator/country;
- user role family, not raw personal data;
- correlation id;
- route/screen id;
- error code when available.

16. App-Specific DD Template

Each app-specific frontend DD must include:

1. Purpose and users.
2. Personas and role matrix.
3. Navigation map.
4. Feature list grouped by workflow.
5. Screen-by-screen flow descriptions.
6. API dependencies by module/DD.
7. State and offline behavior.
8. Error handling.
9. Permissions.
10. Regional/operator configuration.
11. Analytics/observability.
12. Acceptance checklist.

17. Initial Application Boundaries

Recommended first implementation split:

Deployable	Contains
bss-backoffice-web	FE-APP-01 Backoffice, CRM/ticketing modules, dispatch, admin, operational reports.
sales-franchise-app	FE-APP-02 outdoor sales/franchise.
contractor-mobile-app	FE-APP-03 contractor technical mobile.
customer-selfcare-app	FE-APP-04 customer frontoffice.
reporting-portal	FE-APP-05 reporting. Can start inside backoffice if budget requires.

This split avoids too many deployables while respecting real UX differences.

18. Implementation Notes For Developers

Recommended frontend architecture:

- Use a shared component library/design system package.
- Use a shared API client package.
- Use a shared auth/session package.

- Use a shared i18n package.
- Keep app-specific routes and workflows in the app package.
- Keep operator config outside the frontend build artifact where possible.

Suggested repository shape:

```
apps/
  bss-backoffice-web/
  sales-franchise-app/
  contractor-mobile-app/
  customer-selfcare-app/
  reporting-portal/
packages/
  ui/
  auth/
  api-client/
  i18n/
  config/
  telemetry/
```

Build-time config should define environment endpoints. Runtime config should define operator/country/brand/feature behavior.

19. Open Decisions For Review

Decision	Recommendation
Mobile implementation technology	Use PWA first if device APIs/offline needs are satisfied; use native wrapper if background sync/camera/storage constraints require it.
Reporting deployment	Start inside Backoffice if reporting is simple; split into FE-APP-05 when dashboards/export permissions become large.
Multi-operator user support	Support in Backoffice from the start; mobile apps can start single-operator per login.
Offline command scope	Approve per app DD. Start with drafts and evidence capture, then add queued commands where safe.
Translation platform	Use JSON/ICU bundles initially; integrate a translation management system later if needed.

20. Acceptance Checklist

- [] Frontend apps use Keycloak/OIDC.
- [] Navigation and actions are role and feature-flag aware.
- [] Backend authorization remains authoritative.
- [] Operator branding loads from runtime config.
- [] Text is translation-key based.
- [] Money/date/number formatting uses locale and currency config.
- [] API client sends correlation id.
- [] Commands use idempotency keys where retry is possible.
- [] Error model maps backend codes to localized messages.
- [] Offline behavior is explicit for mobile apps.
- [] File upload follows FOUNDATION_FILE_STORAGE.
- [] Telemetry avoids PII and includes app/operator/version/correlation id.